

实验报告：课后练习1

项目	内容
课程名称	系统开发工具基础
实验主题	课程概览 + Shell 入门 (Missing Semester 2026 课后练习)
题目编号	课后练习1
学号	25020007191
姓名	周御基
实验日期	2026年8月31日

一、实验目的

1. 熟悉类 Unix Shell 与基础文件系统命令
2. 掌握 glob 模式、引号、标准流重定向、退出码等 Shell 核心概念
3. 通过实际练习巩固脚本编写、权限设置与管道文本处理

二、实验环境

- 操作系统：Ubuntu Linux (远程服务器 192.168.79.130)
- 命令行工具：Bash
- 远程连接：SSH (paramiko)

三、实验步骤与代码

本练习选自 Missing Semester 2026 第 1 讲「课程概览 + Shell 入门」，共完成 10 道题。

练习1：确认当前 Shell

```
echo $SHELL
```

输出结果：

```
/bin/bash
```

确认使用的是 Bash，符合课程对类 Unix Shell 的要求。

练习2：解读 ls -l 输出的前 10 个字符

```
ls -l / | head -5
```

输出结果：

```
total 2820164
lrwxrwxrwx   1 root root          7 Apr  9  2025 bin → usr/bin
drwxr-xr-x   4 root root      4096 Aug 24 06:33 boot
dr-xr-xr-x   2 root root      4096 Apr 15  2025 cdrom
drwxr-xr-x  20 root root     4120 Aug 23 06:14 dev
```

`ls -l` 每行最前面的 10 个字符表示**文件类型与权限**：

- 第 1 位：文件类型（`-` 普通文件、`d` 目录、`l` 符号链接、`c` 字符设备、`b` 块设备）
- 第 2~4 位：所属用户（owner）的读/写/执行权限（r/w/x）
- 第 5~7 位：所属组（group）的读/写/执行权限
- 第 8~10 位：其他用户（others）的读/写/执行权限

例如 `lrwxrwxrwx` 表示符号链接且所有人都有全权限；`drwxr-xr-x` 表示目录，owner 可读写执行，group 和 others 只读+执行。

练习3：glob 模式

```
mkdir -p ~/prac_glob && cd ~/prac_glob
touch a.txt b.txt c.txt file1.txt file2.txt fileX.txt
ls *.txt
ls file?.txt
ls {a,b,c}.txt
```

输出结果：

```
-- ls *.txt --
a.txt      b.txt      c.txt      file1.txt  file2.txt  fileX.txt
-- ls file?.txt --
file1.txt  file2.txt  fileX.txt
-- ls {a,b,c}.txt --
a.txt     b.txt     c.txt
```

glob 是 Shell 用通配符匹配文件名的模式：`*` 匹配任意多个字符，`?` 匹配恰好 1 个字符，`{a,b,c}` 是花括号展开（列出多个候选项）。

练习4：引号与字面量输出

```
printf 'cost $5 has a bang !\nsecond line\n'
```

输出结果：

```
cost $5 has a bang !
second line
```

- **单引号** `'...'`：完全按字面量处理，不做任何展开（`$`、`!`、变量等都原样输出）。
- **双引号** `"..."`：会发生变量、命令、算术等展开与历史展开（`!` 在交互式 Bash 中可能被历史展开），但 `$`、反引号等特殊字符被保留语义。
- **`$'...'` (ANSI-C 引号)**：允许用 `\n`、`\t` 等转义序列，适合构造含换行符的字符串。

上面的命令用单引号构造了一个同时包含字面量 `$`、`!` 和换行符的字符串并输出。

练习5：标准流重定向

```
ls /nonexistent /tmp > stdout.log 2> stderr.log
ls /nonexistent /tmp > both.log 2>&1
```

输出结果：

```
stdout.log:  /tmp 下的正常输出
stderr.log:  ls: cannot access '/nonexistent': No such file or directory
both.log:    错误信息与正常输出（顺序：先错误后正常）
```

- `>` 重定向 stdout（文件描述符 1），`2>` 重定向 stderr（文件描述符 2）。

- 要把两者合并到同一文件: `> file 2>&1` , 即先把 stdout 指向 file, 再把 stderr 指向 stdout 当前的目标。

练习6: exit status 与 && / || 短路

```
[ -d /tmp/mydir ] || mkdir -p /tmp/mydir && echo "ready"
ls -ld /tmp/mydir
```

输出结果:

```
ready
drwxrwxr-x 2 joe joe 40 Aug 31 08:26 /tmp/mydir
```

`$?` 保存上一条命令的退出码 (0=成功) 。 `||` 仅在前一条失败时执行后一条; `&&` 仅在前一条成功时执行。这里用 `[ -d ... ] || mkdir` 实现"仅当目录不存在时才创建"。

练习7: 编写检查文件是否存在的脚本 check.sh

创建 `check.sh` :

```
#!/usr/bin/env bash
# 检查文件是否存在, 用法 ./check.sh <path>
if [ -f "$1" ]; then
    echo "EXISTS (regular file): $1"
else
    echo "MISSING (not a regular file): $1"
fi
```

运行:

```
chmod +x ~/check.sh
./check.sh ~/check.sh
./check.sh ~/does_not_exist_file
```

输出结果:

```
EXISTS (regular file): /home/joe/check.sh
MISSING (not a regular file): /home/joe/does_not_exist_file
```

脚本通过 `$1` 接收第一个参数, 用 `[ -f ... ]` 判断是否为普通文件并给出相应提示。

练习8: set -x 跟踪执行

创建 `xtest.sh` :

```
#!/usr/bin/env bash
set -x
whoami
echo "hello, current user is $(whoami)"
set +x
echo "done"
```

运行:

```
chmod +x ~/xtest.sh && ./xtest.sh
```

输出结果:

```
joe
hello, current user is joe
done
( stderr 中显示 + 前缀的跟踪行 )
```

`set -x` 让 Shell 在执行每条命令前把该命令打印到 stderr (以 `+` 开头, 可叠加成 `++`), 便于调试, `set +x` 关闭跟踪。

练习9: 复制文件并带当天日期备份名

```
echo "some notes" > ~/notes.txt
cp ~/notes.txt ~/notes_$(date +%Y-%m-%d).txt
ls -l ~/notes.txt ~/notes_$(date +%Y-%m-%d).txt
```

输出结果:

```
-rw-rw-r-- 1 joe joe 11 Aug 31 08:27 /home/joe/notes_2026-08-31.txt
-rw-rw-r-- 1 joe joe 11 Aug 31 08:27 /home/joe/notes.txt
```

用命令替换 `$(date +%Y-%m-%d)` 生成当日日期字符串, `notes.txt` 被复制为 `notes_2026-08-31.txt`。

练习10：找出 home 目录最常见的 5 种文件扩展名

```
find ~ -maxdepth 1 -type f | sed 's/.*\./' | tr 'A-Z' 'a-z' | sort | uniq -c |  
sort -rn | head -5
```

输出结果：

```
10 txt  
3 log  
2 sh  
1 xauthority  
1 viminfo
```

管道拆解：

1. `find ~ -maxdepth 1 -type f`：列出 home 目录一级子路径下的普通文件
2. `sed 's/.*\./'`：截取最后一个 `.` 之后的部分（扩展名）
3. `tr 'A-Z' 'a-z'`：统一转小写（避免 `TXT` 与 `txt` 被分开统计）
4. `sort | uniq -c`：排序后统计每个扩展名出现次数
5. `sort -rn | head -5`：按次数降序取前 5

四、实验结果

4.1 结果展示

完成 10 道 Shell 入门练习，覆盖：Shell 确认、`ls -l` 权限解读、glob 匹配、引号语义、标准流重定向、退出码与短路、脚本参数与 `test`、`set -x` 调试、日期备份名、扩展名统计管道。全部命令在远程 Ubuntu 服务器上执行成功。

4.2 结果分析

1. `ls -l` 前 10 字符 = 文件类型 + 三组 `rwX` 权限，是 Unix 文件权限模型的基础。
2. glob 与正则不同：glob 仅作用于路径名匹配，`* / ? / []` 语义简单直接。
3. 单引号完全字面、双引号会展开变量、`$'...'` 支持转义，三者使用场景不同。
4. 重定向顺序 `> file 2>&1` 与 `2>&1 > file` 结果不同，需理解描述符指向关系。
5. 管道思想（`find→sed→sort→uniq→head`）可复用于大量文本统计场景。

五、实验总结

通过本次实验，我掌握了以下技能：

1. 类 Unix Shell (Bash) 基础命令与 `ls -l` 权限解读
 2. glob 匹配模式与变量展开规则
 3. 单/双引号及 `$' ... '` 引号的差异
 4. 标准流重定向 (`>` 、 `2>` 、 `2>&1`) 与退出码、 `&&` / `||` 短路
 5. 脚本参数 `$1` 、 `test -f` 、 `set -x` 调试、日期命令替换与 `find|sort|uniq|head` 管道
-

实验中的脚本文件：

1. `Scripts/practice1/check.sh`
2. `Scripts/practice1/xtest.sh`